Семейство языков ML

Парадигмы и среда выполнения

Standard ML OCaml F#

Конкурентность, парадигмы, особенности ML, выполнение и память

Основные примеры: OCaml 5.4

Порядок тем

Темы третьей презентации

- Конкурентность и параллелизм: задачи, домены, общее состояние.
- Парадигмы: функции, объекты и контракты.
- Специфические конструкции: образцы, GADT, lazy, эффекты.
- Среды выполнения, сборка мусора и время жизни ресурсов.

Возможность языка и её реализация - разные уровни описания.

Конкурентность и параллелизм

Независимые задачи не обязательно работают одновременно.

- Конкурентность: несколько задач находятся в работе за один период.
- Параллелизм: вычисления выполняются одновременно на разных ядрах.
- Ожидание ввода-вывода можно перекрыть другой работой.
- Для ускорения вычислений нужны ресурсы и достаточно крупные задачи.

Асинхронность сама по себе не обещает ускорения вычислений.

Домены OCaml 5

Domain.spawn запускает вычисление в новом домене.

- Каждый вызов sum использует собственный счётчик.
- Два домена считают разные диапазоны.
- Domain.join ждёт завершения и возвращает результат.

```
let sum first last =
  let total = ref 0 in
  for n = first to last do
    total := !total + n
  done;
  !total
let d = Domain.spawn (fun () -> sum 1 500)
let right = sum 501 1_000
let total = Domain.join d + right
let () = Printf.printf "sum = %d\n" total
```

Результат: 500500. Малый пример показывает механизм, а не ускорение.

src/slide_04_domains.ml

Общее состояние требует координации

Атомарное увеличение защищает общий счётчик.

- Оба домена обновляют одну Atomic-ячейку.
- Atomic.incr увеличивает значение как одну операцию.
- После join все 2000 увеличений завершены.

```
let count = Atomic.make 0
let increment () =
  for _ = 1 to 1_000 do Atomic.incr count done
let d = Domain.spawn increment
let () = increment ()
let () = Domain.join d
let () = Printf.printf "count = %d\n"
  (Atomic.get count)
```

Atomic.get и последующий Atomic.set вместе не образуют атомарное увеличение.

src/slide_05_atomics.ml

Механизмы в трёх языках

Общая задача решается разными средствами.

- OCaml 5.4: домены для параллелизма; эффекты - основа библиотек конкурентности.
- Standard ML: CML - библиотека потоков и синхронного обмена сообщениями.
- F# 9: async, task и библиотеки .NET для асинхронной работы.

Эффект не является готовым планировщиком; async не означает отдельное ядро.

Функциональный стиль

Функции преобразуют значения; эффекты можно вынести наружу.

- Неизменяемый список служит входом вычисления.
- Функция-аргумент описывает один шаг свёртки.
- Сумма возвращается как значение; печать выполняется отдельно.

```
let scores = [3; -1; 5; 0]
let sum_positive xs =
  List.fold_left
    (fun acc x -> if x > 0 then acc + x else acc)
    0 xs
let () = Printf.printf "%d\n" (sum_positive scores)
```

Функциональная основа общая для SML, OCaml и F#; изменяемость разрешена.

src/slide_07_functional.ml

Объекты и классы OCaml

Объект объединяет внутреннее состояние и методы.

- class задаёт класс; new создаёт объект.
- val mutable хранит изменяемое поле экземпляра.
- Методы вызываются через #; inherit позволяет наследование.

```
class counter =
  object
    val mutable value = 0
    method get = value
    method incr = value <- value + 1
  end
let c = new counter
let () =
  c#incr;
  Printf.printf "%d\n" c#get
```

Типы объектов структурные: подходят нужные методы, общий предок не обязателен.

src/slide_08_objects.ml

Контракты: объекты, модули, интерфейсы

Контракт задаёт доступные операции, но механизмы различаются.

- F# interface: именованный контракт с явной реализацией.
- OCaml object type: структурный набор методов.
- SML/OCaml signature: контракт модуля, включая типы и значения.

```
type IPrinter =
    abstract Print : string -> unit
let printer =
    { new IPrinter with
        member _.Print text = printfn "%s" text }
printer.Print "ML"
```

Пример на F# 9. Здесь «протокол» означает договор об операциях.

src/slide_09_interfaces.fsx

Образцы: структура и дополнительные условия

Сопоставление умеет разбирать вложенные данные.

- Some (x, y) раскрывает option и кортеж.
- when добавляет условие после совпадения образца.
- Альтернативы через | объединяют одинаковую обработку.

```
let classify = function
  | Some (x, y) when x = y -> "equal"
  | Some (0, _) | Some (_, 0) -> "on axis"
  | Some _ -> "different"
  | None -> "missing"
let () =
  List.iter (fun x -> print_endline (classify x))
    [Some (2, 2); Some (0, 3); Some (1, 4); None]
```

Выбирается первая подходящая ветвь; Some (0, 0) даст equal.

src/slide_10_patterns.ml

Обычные и полиморфные варианты

Похожие варианты имеют разные правила объявления и типов.

- Red и Green принадлежат объявленному типу light.
- Теги `Red и `Green допустимы без отдельного объявления.
- Вывод типов фиксирует, какие теги принимает функция.

```
type light = Red | Green
let closed = function Red -> "stop" | Green -> "go"
let flexible = function
  | `Red -> "stop"
  | `Green -> "go"
let () =
  print_endline (closed Red);
  print_endline (flexible `Green)
```

flexible принимает только перечисленные теги; `Blue здесь не подходит.

src/slide_11_variants.ml

GADT: тип результата виден в данных

Конструкторы уточняют параметр типа.

- Int создаёт int expr, a Bool - bool expr.
- Add принимает только целочисленные выражения.
- В eval каждая ветвь уточняет тип результата.

```
type _ expr =
  | Int : int -> int expr
  | Bool : bool -> bool expr
  | Add : int expr * int expr -> int expr
let rec eval : type a. a expr -> a = function
  | Int n -> n
  | Bool b -> b
  | Add (x, y) -> eval x + eval y
let expression = Add (Int 20, Int 22)
let () = Printf.printf "%d\n" (eval expression)
```

Add (Int 1, Bool true) отклоняется до выполнения программы.

src/slide_12_gadt.ml

lazy: вычислить позже и запомнить

Отложенное вычисление отличается от обычной функции.

- lazy сохраняет вычисление, не выполняя его сразу.
- Первый Lazy.force вычисляет и запоминает результат.
- Повторный force использует сохранённое значение.

```
let calls = ref 0
let answer = lazy (incr calls; 21 * 2)
let first = Lazy.force answer
let second = Lazy.force answer
let () =
  Printf.printf "%d %d; calls = %d\n"
    first second !calls
```

Вывод: 42 42; calls = 1. Конкурентный Lazy.force требует синхронизации.

src/slide_13_lazy.ml

Эффект может возобновить вычисление

perform передаёт управление обработчику вместе с продолжением.

- Ask объявлен как операция, возвращающая int.
- Обработчик возобновляет k со значением 41.
- Затем выполняется оставшееся + 1; итог равен 42.

```
type _ Effect.t += Ask : int Effect.t
let answer =
  try Effect.perform Ask + 1 with
  | effect Ask, k -> Effect.Deep.continue k 41
let () = Printf.printf "%d\n" answer
```

Это не отдельный поток. Необработанный эффект вызывает Effect.Unhandled.

src/slide_14_effects.ml

OCaml: байткод или машинный код

Один исходник поддерживает два основных пути компиляции.

- ocamlc создаёт байткод; его выполняет ocamlrun.
- ocamlopt создаёт машинный код целевой платформы.
- Оба пути используют runtime, включая сборку мусора.

```
let backend =
  match Sys.backend_type with
  | Sys.Native -> "native"
  | Sys.Bytecode -> "bytecode"
  | Sys.Other name -> name
let () = print_endline backend
```

Вывод: bytecode после ocamlc, native после ocamlopt.

src/slide_15_backend.ml

У семейства нет общей виртуальной машины

Среда выполнения зависит от языка и реализации.

- SML ’97 задаёт язык; способ исполнения выбирает реализация.
- MLton компилирует программу целиком и создаёт исполняемый файл.
- F# для .NET компилируется в IL; CLR обеспечивает выполнение.
- В обычном пути .NET JIT переводит IL в машинный код.

Сходство синтаксиса ML не означает одинаковый байткод или формат библиотек.

Сборка мусора в OCaml 5

GC освобождает память по достижимости объектов.

- Небольшие новые объекты обычно попадают в minor-кучу своего домена.
- Пережившие minor-сборку данные переходят в major-кучу.
- Домены участвуют в сборке; major-сборка чередуется с работой программы.
- Достижимые данные сохраняются, даже если больше не нужны приложению.

Автоматическое освобождение памяти не исключает её чрезмерного удержания.

Достижимость, замыкания и общие данные

Два замыкания могут сохранять один изменяемый объект.

- Массив создан внутри make_access.
- read и write обращаются к нему после возврата.
- Изменение через write видно через read: результат 7.

```
let make_access () =
  let data = [|0; 1; 2|] in
  ((fun () -> data.(0)), (fun x -> data.(0) <- x))
let read, write = make_access ()
let () = write 7
let () = Printf.printf "%d\n" (read ())
```

Пока замыкание сохраняет нужную ссылку, объект остаётся достижимым.

src/slide_18_reachability.ml

Ресурсы имеют отдельное время жизни

Файловый ресурс закрывают в определённой точке программы.

- with_open_text закрывает канал и при исключении callback.
- Fun.protect выполняет очистку при выходе из callback.
- В F# сходную задачу решает use для IDisposable.

```
let path = Filename.temp_file "ml-" ".txt"
let () =
  Fun.protect ~finally:(fun () -> Sys.remove path)
    (fun () ->
      Out_channel.with_open_text path (fun oc ->
        output_string oc "hello\n");
      let text = In_channel.with_open_text path
        In_channel.input_all in
      print_string text)
```

GC управляет памятью; своевременное освобождение ресурса задаёт программа.

src/slide_19_resources.ml

Главные выводы и источники

Типы, модель выполнения и ресурсы нужно рассматривать вместе.

- Домены дают параллелизм; общие изменяемые данные требуют координации.
- Функции, объекты и модули по-разному организуют код.
- Обработчики эффектов расширяют управление потоком.
- GC освобождает память; время жизни файлов и сокетов задаёт программа.

OCaml 5.4: параллельное программирование

ocaml.org/manual/5.4/parallelism.html

OCaml 5.4: обработчики эффектов

ocaml.org/manual/5.4/effects.html

OCaml 5.4: Gc

ocaml.org/manual/5.4/api/Gc.html

The Definition of Standard ML, Revised

smlfamily.github.io/sml97-defn.pdf

Microsoft: Common Language Runtime

learn.microsoft.com/en-us/dotnet/standard/clr

Источники: OCaml 5.4 Manual, SML ’97 Definition, документация F#/.NET.